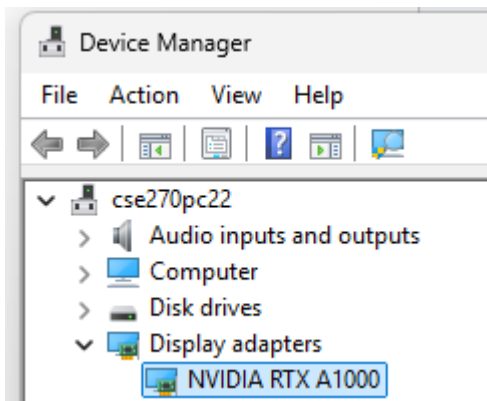


21/05/2026 (Antonio)

Setup

- CSE/066 being used for an exam
 - In any case, if they have NVIDIA Quadro P1000 cards, as I saw on Slack, those only have 4GB VRAM which is far too little to run local LLMs.
- Went to CSE/270
 - CSE/270 has RTX A1000s:



The [specs](#) say that these cards have 8GB GDDR6, which is enough for small local LLMs.

Tried running [Ollama](#) from Git Bash (with GPU integration) and [OpenWebUI](#):

```
docker run -d --gpus=all -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main
```

Both images pulled correctly and started OK (OpenWebUI had to download some files, and then it became available via <http://localhost:3000/>). Ollama logs showed the A1000 was picked up:

```
time=2026-05-21T11:51:53.684Z level=INFO source=types.go:42 msg="inference compute" id=GPU-e241c696-47c7-c33a-3577-68c9ef6aff5e filter_id="" library=CUDA compute=8.6 name=CUDA0 description="NVIDIA RTX A1000" libdirs=ollama,cuda_v13 driver=13.2 pci_id=0000:01:00.0 type=discrete total="8.0 GiB" available="7.3 GiB"
```

Evaluation approach

Pulled models into Ollama (best done from Docker Desktop's Exec tab - you can pull via OpenWebUI but it will cancel the download if you close the tab):

- mistral:7b
- devstral-small-2 (it's 24B)
- gemma4:26b
- gemma4:e4b
- granite4.1:8b

- qwen3.5:9b
- olmo-3:7b-instruct
- gpt-oss:20b
- deepseek-coder-v2:16b

After pulling all these models, the ollama Docker volume had grown to 106.3GB.

Tried asking for a generic software engineering prompt that would check their training sets, with no guidance:

Please summarise the most important structural software design patterns from the Gang of Four book.

Results (raw responses below)

Model	Response OK?	Output tokens/second (as reported via OpenWebUI)
mistral:7b	✅ had all 7 patterns	36.24
gemma4:e4b	😞 missed Flyweight	25.23
devstral-small-2	❌ model requires more system memory (9.5 GiB) than is available (6.4 GiB)	
Gemma 4 26B MoE A4B	❌ model requires more system memory (11.8 GiB) than is available (5.7 GiB)	
Phi4:14B	✅ had all 7 patterns	10.95
Granite 4.1 8B	✅ had all 7 patterns	30.43
Qwen 3.5 9B	😞 had all 7 patterns, but spent 3 minutes thinking (!)	17.28
Olmo 3 7B Instruct	😞 two of the patterns are incorrect, some odd descriptions	32.07
gpt-oss:20b	❌ model requires more system memory (6.7 GiB) than is available (5.0 GiB)	
deepseek-coder-v2:16b	😞 mentions creational patterns in addition to the structural ones	30.25

LLM responses

Mistral:7b

The Gang of Four (GoF) book, also known as "Design Patterns: Elements of Reusable Object-Oriented Software," identifies 23 design patterns that address common problems in software design. Here's a summary of some important structural patterns:

1. ****Adapter****: Allows classes with incompatible interfaces to work together by wrapping an existing class and providing a new interface that the client expects.

2. **Bridge**: Decouples an abstraction from its implementation, allowing the two to vary independently. This enables easier maintenance, extension, and reuse.
3. **Composite**: Composes objects into tree structures to represent part-whole hierarchies. It allows clients to treat individual objects and compositions of objects uniformly.
4. **Decorator**: Attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.
5. **Facade**: Provides a simplified interface to a complex system, making it easier for clients to use the system without needing detailed knowledge of its underlying workings.
6. **Flyweight**: Aims to minimize memory usage by sharing common objects in a way that doesn't affect their external behavior. It does this by representing fine-grained internal state outside the object, and only storing coarse-grained state within the object itself.
7. **Proxy**: Provides a surrogate or placeholder for another object to control access to it, reduce its load, or enable customization. Proxies are often used for remote objects, lazy loading, and security.

Gemma E4B

The Gang of Four (GoF) book, *Design Patterns: Elements of Reusable Object-Oriented Software*, is foundational to software design. When discussing "structural patterns," we are referring to patterns that concern how classes and objects are composed to form larger structures, thereby simplifying the design.

Here is a summary of the most important structural design patterns from the GoF book, grouped by their primary function.

Summary of Important Structural Design Patterns (GoF)

1. Composition and Assembly Patterns (Structuring Components)

These patterns are crucial for building complex systems out of simpler, interchangeable parts.

★ Adapter

* **Problem**: You have existing classes that use interfaces incompatible with the new component you want to use.

* **Solution**: The Adapter pattern acts as a wrapper or translator. It converts the interface of a class into another interface that clients expect.

* **Analogy**: A power adapter that allows a device built for one electrical standard (e.g., US) to plug into another (e.g., UK).

* **Key Use**: Integrating third-party libraries or legacy code into a modern system without modifying the existing code.

★ Decorator

* **Problem**: You need to add responsibilities (behaviors) to an object dynamically and transparently, without modifying its original class structure.

* **Solution**: The Decorator pattern wraps the object, and the decorator itself implements the same interface as the object it wraps. It adds functionality *before* or *after* delegating the call to the wrapped component.

* **Analogy**: Ordering coffee. The base object is the coffee. The decorations (milk, whipped cream, syrup) are added one by one, each adding new functionality (taste, volume, richness).

* **Key Use:** Adding optional or layered responsibilities (e.g., adding logging, compression, or authorization to a basic service call).

* **Contrast to Inheritance:** Decorator is preferred over inheritance because it allows for a much more flexible and runtime-pluggable combination of behaviors.

★ Composite

* **Problem:** You have a tree-like structure (a composition of parts) where some nodes are single units (leaves) and others are complex groupings (composites). Clients should be able to treat both leaves and composites uniformly.

* **Solution:** The Composite pattern treats individual objects (leaves) and groups of objects (composites) uniformly through a common interface. The composite object simply delegates requests to its child objects.

* **Analogy:** A file system. Individual files are leaves. Folders are composites. When you request to list contents, the parent folder iterates through its children, treating all types uniformly.

* **Key Use:** Modeling hierarchical structures, such as organizational charts, UI component trees, or abstract syntax trees (ASTs) in compilers.

2. Simplifying Interfaces and Dependencies (Managing Interfaces)

These patterns help reduce coupling and make the system more modular.

★ Facade

* **Problem:** A subsystem is composed of many complex classes and interfaces that are difficult for a client to use directly.

* **Solution:** The Facade pattern provides a single, high-level interface (a "facade") to a complex subsystem. It shields the client from the intricate dependencies and internal complexity of the subsystem.

* **Analogy:** A smart TV remote control. Instead of needing to understand the networking protocols, HDMI inputs, and internal logic of the TV (the subsystem), you just press "Open Netflix."

* **Key Use:** Simplifying access to complex libraries, microservices, or large, interconnected backend modules.

★ Bridge

* **Problem:** Your design has two or more dimensions of variability. For example, a "Vehicle" might vary by *type* (Car, Truck) and *interface* (Electric, Gas). Combining them with deep inheritance leads to a combinatorial explosion (many subclasses).

* **Solution:** The Bridge pattern decouples the abstraction (the *what*—e.g., Car) from its implementation (the *how*—e.g., Gas Engine). Both are defined by separate interfaces, allowing them to be combined at runtime.

* **Analogy:** A modern laptop. The abstraction is the *device* (the laptop). The implementation is the *power source* (the battery vs. the external adapter). You can change the power source without changing the fundamental laptop logic.

* **Key Use:** Designing systems where component functionality needs to be interchangeable and extendable along multiple orthogonal axes.

3. Utilizing Object Relationships (Building Frameworks)

★ Proxy

* **Problem:** A client needs to access an object, but needs to control that access in some way (e.g., restricting access, adding logging, or implementing lazy loading).

* **Solution:** The Proxy pattern provides a stand-in object (the proxy) that holds a reference to the real object. The proxy implements the same interface as the real object but intercepts method calls to perform auxiliary duties *before* or *after* delegating to the real object.

* **Analogy:** A building concierge. The concierge (proxy) stands in front of the penthouse (real object). When you ask for the penthouse, the concierge first checks your ID, verifies your appointment, and logs your entry (proxy duties) before allowing you access.

* **Variations:**

* **Remote Proxy:** Used to represent a service object that resides in a different address space (e.g., calling a Web Service).

* **Virtual Proxy:** Used for lazy loading, deferring the creation of an expensive object until the first time it is actually requested.

* **Key Use:** Implementing security mechanisms, access control, logging, caching, and lazy loading.

🧠 Conceptual Summary / When to Use Which Pattern

| Pattern | Goal / Purpose | What it Does | When to Use It |

| :--- | :--- | :--- | :--- |

| **Decorator** | Dynamic Behavior Addition | Wraps an object to add responsibilities. | When you need to add *optional* or *layered* behaviors at runtime (e.g., adding security features to an existing service). |

| **Composite** | Uniform Structure Handling | Treats individual items and groups of items identically. | When modeling hierarchical structures (e.g., UI components, file systems). |

| **Facade** | Simplifying Access | Provides a single, simplified entry point to a complex subsystem. | When a client must interact with many complex, interconnected classes, but you only want them to use a simple interface. |

| **Adapter** | Interoperability / Translation | Changes the interface of one class to match the expectations of another. | When integrating disparate systems or third-party libraries with incompatible interfaces. |

| **Proxy** | Controlling Access / Decoupling | Provides a stand-in object to manage access or defer creation. | When you need control over how an object is accessed (e.g., lazy loading, security checks, remote calls). |

| **Bridge** | Decoupling Abstraction/Implementation | Separates an object's core conceptual behavior from its underlying implementation details. | When you have multiple dimensions of variation, preventing the need for an unmanageable number of inherited subclasses. |

Phi4:14B

The "Gang of Four" (GoF) book, titled *Design Patterns: Elements of Reusable Object-Oriented Software*, is a seminal work that outlines 23 classic software design patterns categorized into three groups: Creational, Structural, and Behavioral patterns. Here's a summary of the most important structural design patterns from the book:

1. **Adapter Pattern**:

- **Purpose**: Allows incompatible interfaces to work together.
- **Structure**: Acts as a bridge between two incompatible interfaces by wrapping an existing class with a new interface so that it becomes compatible with another class.

2. **Bridge Pattern**:

- **Purpose**: Decouples abstraction from implementation, allowing them to vary independently.
- **Structure**: Comprises an abstract class (abstraction) and a separate interface for implementation. The concrete classes of both can be combined in various ways.

3. **Composite Pattern**:

- **Purpose**: Allows you to compose objects into tree structures to represent part-whole hierarchies.
- **Structure**: Enables clients to treat individual objects and compositions uniformly by defining an interface common to both simple (leaf) elements and composite elements.

4. **Decorator Pattern**:

- **Purpose**: Dynamically adds behavior or responsibilities to individual objects without affecting other instances of the same class.

- **Structure**: Involves a set of decorator classes that are used to wrap concrete components. Decorators provide additional functionality while maintaining the interface of the original component.

5. **Facade Pattern**:

- **Purpose**: Provides a simplified interface to a complex subsystem, reducing complexity for clients.
- **Structure**: Introduces a facade class that abstracts the complexities of a larger system and provides a simpler interface.

6. **Flyweight Pattern**:

- **Purpose**: Minimizes memory usage or computational expenses by sharing as much data as possible with related objects.
- **Structure**: Separates intrinsic (shared) state from extrinsic (non-shared) state so that common states can be shared among multiple objects, reducing resource consumption.

7. **Proxy Pattern**:

- **Purpose**: Provides a surrogate or placeholder for another object to control access to it.
- **Structure**: Involves a proxy class with an interface identical to the subject it represents, which controls access to the real subject and may add additional behavior such as lazy initialization or access control.

These structural patterns are crucial in software design because they help manage relationships between objects and classes, enabling more flexible, reusable, and maintainable code architectures.

Granite4.1:8b

The "Design Patterns: Elements of Reusable Object-Oriented Software" book, authored by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (known as the Gang of Four or GoF), introduced a collection of 23 fundamental software design patterns categorized into three main groups: **Creational**, **Structural**, and **Behavioral** patterns. Below is a summary of the most important Structural Design Patterns from this seminal work:

1. Adapter Pattern

- **Purpose**: Allows incompatible interfaces to work together by converting the interface of a class into another interface expected by the client.
- **Key Idea**: Wraps an existing class with a new interface without altering its structure, facilitating interaction between disparate classes.

2. Bridge Pattern

- **Purpose**: Separates abstraction from implementation so that both can vary independently.
- **Key Idea**: Defines an interface for controlling the associated abstraction, allowing the abstraction and its implementation to be modified independently.

3. Composite Pattern

- **Purpose**: Composes objects into tree structures to represent part-whole hierarchies, letting clients treat individual objects and compositions uniformly.
- **Key Idea**: Treats a group of objects as if it were a single object, enabling recursive composition for complex hierarchical structures (e.g., file systems or organizational charts).

4. Decorator Pattern

- **Purpose**: Attaches additional responsibilities to an object dynamically without affecting other objects from the same class.
- **Key Idea**: Provides a flexible alternative to subclassing for extending functionality by wrapping the original object with a decorator that adds new behavior.

5. Facade Pattern

- **Purpose:** Provides a unified interface to a set of interfaces in a subsystem, making it easier to use than managing multiple individual interfaces directly.
- **Key Idea:** Offers a simplified and consistent entry point for clients, hiding the complexities of the subsystem behind a single, higher-level interface.

6. Flyweight Pattern

- **Purpose:** Minimizes memory usage by sharing as much data as possible with similar objects rather than storing it repeatedly.
- **Key Idea:** Uses shared objects to support large numbers of fine-grained objects efficiently, particularly when most object state can be made extrinsic (i.e., externalized).

7. Proxy Pattern

- **Purpose:** Provides a surrogate or placeholder for another object to control access to it.
- **Key Idea:** Controls the creation and interaction with an expensive-to-create or protected resource by routing all calls through a proxy object.

These Structural Design Patterns are pivotal in creating flexible, maintainable, and scalable software architectures. They address common challenges such as interface compatibility, complexity management, extensibility, and resource optimization, thereby enhancing code reusability and adaptability to changing requirements.

Qwen3.5:9B

In the *Gang of Four* (GoF) book, *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), design patterns are categorized into three main groups: Creational, Structural, and Behavioral.

The **Structural Patterns** focus on **object composition**—how objects are assembled to form larger structures. The core goal is to simplify architecture, promote loose coupling, and allow for the creation of complex systems without hardcoding dependencies.

There are **seven** structural patterns in the original book. Below is a summary of all of them, highlighting their purpose and primary use cases.

1. Adapter

- The Purpose:** To make incompatible interfaces work together.
- Concept:** An "Adapter" acts as a translator. It allows a client to use a class whose interface it doesn't expect.
- Analogy:** A power plug adapter. You have a device with a specific plug type, and you use an adapter to plug it into a different outlet.
- Key Takeaway:** Essential when integrating third-party libraries where you cannot change the source code.

2. Bridge

- The Purpose:** To decouple an abstraction from its implementation.
- Concept:** This pattern uses two separate hierarchies: the **Abstraction** hierarchy and the **Implementor** hierarchy. A high-level interface does not know the details of the implementation; instead, it relies on the bridge object.
- Analogy:** A car engine (implementation) and a chassis (abstraction) that can be swapped out for different engines or chassis types without redesigning the connection.
- Key Takeaway:** Use this when you want to vary both the high-level structure and the implementation independently.

3. Composite

****The Purpose:**** To allow objects to be composed into tree structures and treat individual objects and compositions uniformly.

* ****Concept:**** A client can talk to a simple object (like a file) or a complex object (like a directory of files) exactly the same way.

* ****Analogy:**** A box of LEGO bricks. You can build one brick or a whole castle, but the interface to "add a brick" remains the same.

* ****Key Takeaway:**** Best for tree structures (file systems, org charts) where recursion is needed.

4. Decorator

****The Purpose:**** To add responsibilities dynamically to individual objects.

* ****Concept:**** Instead of using inheritance to add new behaviors (e.g., adding encryption, caching, or logging), you wrap the object in other objects (decorators). Each decorator adds a specific feature.

* ****Analogy:**** Adding toppings to a pizza or layers to a dress. You can add a "cheese" layer, then a "pepperoni" layer without rewriting the dough code.

* ****Key Takeaway:**** Crucial for frameworks where you need to build up behaviors without creating classes for every possible combination.

5. Facade

****The Purpose:**** To provide a simplified interface to a complex subsystem.

* ****Concept:**** A single class (the Facade) provides a high-level interface to a set of interfaces in a subsystem. It hides the complexity of the underlying subsystem.

* ****Analogy:**** A hotel reception desk. You only talk to the receptionist (Facade) to handle booking a room, checking out, etc., rather than navigating through every single department (housekeeping, front desk, maintenance, security).

* ****Key Takeaway:**** Use this to simplify usage of a large, complex codebase or library.

6. Flyweight

****The Purpose:**** To minimize memory usage by sharing data.

* ****Concept:**** It separates an object into a ****Shared**** part (intrinsic state, shared by multiple contexts) and an ****Unique**** part (extrinsic state, specific to the context).

* ****Analogy:**** A factory making thousands of paper cups. The mold is one shared object (Flyweight), but each cup gets a different handle color (Extrinsic state).

* ****Key Takeaway:**** Essential for high-performance applications (like video games or rendering engines) where you have many similar objects (e.g., thousands of identical sprites or textures).

7. Proxy

****The Purpose:**** To provide a surrogate or placeholder for another object to control access.

* ****Concept:**** The Proxy object holds the reference to the real object but controls access (security, lazy initialization, or logging).

* ****Analogy:**** A hotel room key or a receptionist. The receptionist controls who gets to enter a VIP room, or you check into a hotel online (lazy initialization) before you ever physically go there.

* ****Key Takeaway:**** Critical for implementing access control, security, or lazy loading of expensive objects.

Summary Comparison

* ****Adapter**** handles **mismatched** interfaces (different shapes).

* ****Bridge**** handles **separation** of abstraction and implementation.

* ****Composite**** handles **tree structures** and uniform treatment.

* ****Decorator**** handles **dynamic behavior addition** via wrapping.

* ****Facade**** handles **simplification** of complex subsystems.

* ****Flyweight**** handles **memory optimization** through sharing.

* **Proxy** handles **access control** and initialization.

These patterns form the foundation of robust, maintainable software architecture. While Behavioral patterns (like Observer or Strategy) focus on interaction and algorithms, Structural patterns focus on **how objects are put together**.

Note: The "Builder" pattern is often classified as Creational (though it deals with building complex objects structurally, the focus is on creation flow), but it is not typically grouped with these 7.*

I hope this helps you understand the Structural patterns from the Gang of Four book! If you need examples or code for any of these, feel free to ask!

Olmo3:7b-instruct

Certainly! The **"Design Patterns: Elements of Reusable Object-Oriented Software"** by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (the "Gang of Four") is a foundational text in software design. While not all patterns are structural (some are creational, behavioral, etc.), a subset of them focus specifically on the organization and composition of objects and classes—often referred to as **structural patterns**. These patterns help in structuring classes and objects to compose them in new ways.

Below is a summary of the most important structural design patterns from the GoF book, along with a brief description of each:

1. **Adapter (Wrapper) Pattern**

- **Purpose:** Allows an object to interface with different clients by adapting its interface to that of another class.
- **Use Case:** When there is a mismatch between the interface of a class and the clients that use it.
- **Example:** A SQL database interface for a user expecting an ORMs interface.

2. **Composite Pattern**

- **Purpose:** Allows treating a group of objects as a single object to simplify the hierarchy.
- **Use Case:** To flatten or simplify the object tree structure so that clients don't need to know the difference between composite and leaf objects.
- **Example:** File systems where directories are composed of files and subdirectories.

3. **Decomposition (also known as "Composite", but sometimes as "Single Responsibility" in terms of splitting objects)**

- **Note:** While not a formal pattern name in GoF, decomposition (breaking a complex class into simpler ones) is an underlying principle that the Composite pattern exemplifies by reorganizing structure.

4. **Proxy Pattern**

- **Purpose:** Provides a surrogate or placeholder for another object, allowing indirect access to it.
- **Use Case:** To add responsibilities (such as logging, security, or virtualization) without modifying the real object.
- **Example:** A remote object that acts as a local proxy to a remote server.

5. **Facade Pattern**

- **Purpose:** Provides a simplified interface to a complex subsystem.
 - **Use Case:** To hide the complexities of a subsystem from the users of the system.
 - **Example:** A user interface that abstracts away the complexities of database operations.
-

6. **Flyweight Pattern**

- **Purpose:** Allows sharing of data among multiple objects to reduce memory usage.
 - **Use Case:** When many very similar objects can be used by sharing common data among them.
 - **Example:** Graphics programs with many reusable symbols that share the same properties.
-

7. **Memento Pattern**

- **Purpose:** Captures and externalizes an object's internal state so that the object can be restored to that state later (without exposing its internals).
 - **Use Case:** Undo/redo functionality, saving configuration states.
 - **Note:** More of a state pattern but often considered structural when dealing with object state encapsulation.
-

8. **Proxy/Decorator (sometimes overlaps with Adapter)**

- **Decorator (or Wraps):** Attaches additional responsibilities to an object dynamically.
 - **Use Case:** Extending the behavior of a core object without subclassing it.
 - **Example:** Adding logging to any class without modifying its code.
-

Summary Table

Pattern Name	Main Purpose	Typical Usage
Adapter	Interface conversion/matching	API compatibility, legacy systems
Composite	Group objects as a single entity	Hierarchical data structures
Proxy	Indirectly manage or control another object	Access control, caching, virtualization
Facade	Simplify complex subsystem interface	External APIs, frameworks
Flyweight	Share state among many similar objects	GUI, game engines, data caching
Memento (State)	Save and restore an object's internal state	Undo/Redo, settings persistence
Decorator (Wrapper)	Dynamically add behavior to an object	Extending functionality without subclass

**Most Important Structural Patterns (GoF Focus):**

The **Adapter**, **Composite**, **Proxy**, **Facade**, **Flyweight**, and **Decorator** are the most frequently cited and practically significant structural patterns from the GoF book. They focus on **organizing and composing objects**, making them reusable and easier to maintain.

If you want to focus strictly on patterns that rearrange or compose objects (not about behavior or state), the primary ones are **Adapter**, **Composite**, **Proxy**, and **Flyweight**.

Let me know if you'd like examples or more detailed explanations for any of these!

deepseek-coder-v2:16b

The Gang of Four (GoF) book, titled "Design Patterns: Elements of Reusable Object-Oriented Software," is a seminal work in software engineering that introduces 23 design patterns organized into several categories. These patterns are categorized by their intent and described with an overall structure, which can be classified as follows:

1. **Creational Patterns**: These patterns focus on object creation mechanisms to create objects in a way suitable for the given situation. There are five creational patterns:

- **Singleton Pattern**: Ensures that only one instance of a class is created and provides a global point of access to it.
- **Factory Method Pattern**: Defines an interface for creating an object, but lets subclasses decide which class to instantiate.
- **Abstract Factory Pattern**: Provides an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Builder Pattern**: Separates the construction of a complex object from its representation so that the same construction process can create different representations.
- **Prototype Pattern**: Specifies the kind of objects to create using a prototypical instance, and creates new objects by copying this prototype.

2. **Structural Patterns**: These patterns deal with class and object composition. There are seven structural patterns:

- **Adapter Pattern**: Converts the interface of a class into another interface clients expect. It lets classes work together that could not otherwise because of incompatible interfaces.
- **Bridge Pattern**: Separates an abstraction from its implementation, allowing both to vary independently.
- **Composite Pattern**: Composes objects into tree structures to represent part-whole hierarchies. Allows clients to treat individual objects and compositions of objects uniformly.
- **Decorator Pattern**: Dynamically adds responsibilities to objects without affecting other objects within the same class. It is often used to extend the functionalities of classes in a flexible manner, similar to subclassing but with more flexibility since it allows you to add or modify behavior at runtime.
- **Facade Pattern**: Provides a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.
- **Flyweight Pattern**: Uses sharing to support large numbers of fine-grained objects efficiently.
- **Proxy Pattern**: Provides a surrogate or placeholder for another object to control access to it.

These patterns are categorized based on their primary purpose and help in creating more flexible, maintainable, and efficient software designs. They provide tested, proven ways to solve common problems in software design and are widely used in the industry.